

DCS602 – Homework 2: Complete Study Guide

University of Buea · Department of Computer Science

Table of Contents

1. [Dataset Overview](#)
 2. [Data Cleaning & EDA](#)
 3. [Task 1 – Basic Bubble Chart](#)
 4. [Task 2 – Interactive Widgets](#)
 5. [Task 3 – Linked Brushing](#)
 6. [Task 4 – Details on Demand](#)
 7. [Information Visualization Theory \(Exam Notes\)](#)
 8. [How to Run Everything](#)
-

1. Dataset Overview

The **CIA World Factbook** dataset (`factbook_-_factbook.csv`) contains **149 countries × 29 attributes**. The attributes span:

Category	Examples
Demographics	Birth rate, Death rate, Population, Life expectancy
Economics	GDP, GDP per capita, GDP real growth rate, Inflation rate
Trade	Exports, Imports, Current account balance
Infrastructure	Highways, Railways, Electricity production/consumption
Energy	Oil consumption, Natural gas consumption
Society	Unemployment rate, Internet users, Military expenditures

Why clean the data first?

Many numeric columns were stored as **strings** with currency symbols and commas, e.g. "\$29,400.00" or "127,417,244". Python's pandas reads these as `object` (text) dtype, so arithmetic and visualisation fail until we strip the formatting and convert to `float`.

2. Data Cleaning & EDA (`eda_and_preparation.py`)

What the script does

Step 1 – Load raw CSV

```
df_raw = pd.read_csv("factbook_-_factbook.csv")
```

Step 2 – Identify problematic columns

Any column whose dtype is `object` but *looks* numeric is a candidate.

Examples: `GDP per capita` (\$29,400.00), `Population` (127,417,244),
`Area` (377,835).

Step 3 – Clean with a helper function

```
def clean_numeric(series):
    s = series.astype(str).str.strip()
    s = s.str.replace(r'[\$,]', '', regex=True) # remove $ and commas
    s = s.str.replace(r'\s.*', '', regex=True) # drop trailing text
    return pd.to_numeric(s, errors='coerce') # NaN for unparseable
```

`errors='coerce'` means values that truly cannot be converted become `NaN` (not an error), which is the correct behaviour for missing data.

Step 4 – Check coverage of the four key columns

All four columns used in Task 1 have **100% coverage** (no missing values),
so no rows need to be dropped for the core tasks.

Step 5 – Save `factbook_clean.csv`

All task scripts load this pre-cleaned file, keeping the cleaning logic
in one place (separation of concerns).

3. Task 1 – Basic Bubble Chart (`p2_bubbles.py`)

What is a bubble chart?

A **scatter plot** where each point is replaced by a circle (bubble)
whose size encodes a third variable. Add color for a fourth variable.
Total: **4 data dimensions in 2D space**.

Visual channel	Variable	Reason for choice
x (horizontal position)	GDP per capita	Economic output per person
y (vertical position)	Life expectancy at birth	Quality-of-life indicator
Circle area	Population	Area maps to "magnitude" intuitively
Circle color	Birth rate	Sequential color scale fits ordered data

Key implementation: radius scaling

Perceptually, humans judge **area**, not radius. So:

If $\text{radius} \propto \text{population} \rightarrow \text{area} = \pi \cdot r^2 \propto \text{population}^2$

This would massively exaggerate large countries. The fix:

```
pop_sqrt = np.sqrt(df["Population"].values)
# Normalise sqrt-population into the range [6, 50] pixels
radii = 6 + (50 - 6) * (pop_sqrt - pop_sqrt.min()) / (pop_sqrt.max() - pop_sqrt.min())
```

Now $\text{radius} \propto \sqrt{\text{population}}$, so $\text{area} \propto \text{population}$. ✓

Color mapper

```
from bokeh.palettes import YlOrRd9
from bokeh.models import LinearColorMapper

mapper = LinearColorMapper(
    palette = list(reversed(YlOrRd9)), # yellow=low, red=high
    low     = df["Birth rate"].min(),
    high    = df["Birth rate"].max(),
)
```

LinearColorMapper maps a numeric range linearly onto a color palette.

We reverse **YlOrRd9** so low birth rate → cool/pale and high → warm/red.

ColumnDataSource

```
from bokeh.models import ColumnDataSource

source = ColumnDataSource(dict(
    x      = df["GDP per capita"].values,
    y      = df["Life expectancy at birth"].values,
    radii   = radii,
    color_val = df["Birth rate"].values,
    country = df["Country"].values,
    ...
))
```

ColumnDataSource is Bokeh's data container. It syncs between the Python backend and the JavaScript frontend. String columns (like **country**) are accessible in tooltips via **@country**.

HoverTool

```
from bokeh.models import HoverTool

hover = HoverTool(tooltips=[
    ("Country", "@country"),
    ("GDP per capita", "@x{$0,0}"),
    ("Life exp", "@y{0.0} yrs"),
    ("Population", "@population{0,0}"),
    ("Birth rate", "@birth_rate{0.00}"),
])
p.add_tools(hover)
```

`@column` refers to a column in the `ColumnDataSource`. The format string `{$0,0}` means: dollar sign prefix, comma-separated thousands, 0 decimal places.

Output

This task produces a **standalone HTML file** (`p2_bubbles.html`) using:

```
output_file("p2_bubbles.html")
show(p)
```

No server needed – open the HTML in any browser.

4. Task 2 – Interactive Widgets (`p2_widgets.py`)

Why widgets?

Task 1 hard-coded the four attribute mappings. But what if the user wants to explore GDP vs Inflation, or compare Death rate to Infant mortality? Widgets give the user **runtime control** over the mapping.

Widgets used

Select (drop-down menu)

```
from bokeh.models import Select

sel_x = Select(title="X axis", value="GDP per capita",
               options=NUMERIC_COLS, width=220)
```

`options` is the list of all numeric column names. `value` is the currently selected item.

Slider

```
from bokeh.models import Slider

slider = Slider(title="Radius scale", value=1.0,
                start=0.2, end=4.0, step=0.1, width=460)
```

Lets the user shrink or enlarge all bubbles for readability.

The callback pattern

```
def update(attr, old, new):
    # Read current widget values
    x_col = sel_x.value
    y_col = sel_y.value
    r_col = sel_r.value
    c_col = sel_c.value
    scale = slider.value

    # Rebuild data dict and push to source
    src.data = make_source(x_col, y_col, r_col, c_col, scale)

    # Update axis labels, title, color bar
    p.xaxis.axis_label = x_col
    p.yaxis.axis_label = y_col
    p.title.text = f"{x_col} vs {y_col}"
    mapper.low = float(np.nanmin(src.data["color_val"]))
    mapper.high = float(np.nanmax(src.data["color_val"]))
    color_bar.title = c_col

    # Register the callback on every widget
    for w in [sel_x, sel_y, sel_r, sel_c, slider]:
        w.on_change("value", update)
```

`on_change("value", callback)` means: whenever this widget's `value` property changes, call `callback(attr, old, new)`.

Assigning `src.data = new_dict` triggers a **full browser re-render** of every glyph that references `src`.

Running as a Bokeh server app

Tasks 2-4 **must** run as server apps because the Python callbacks need a live Python process:

```
bokeh serve --show p2_widgets.py
```

Bokeh starts a local web server, opens the app in your browser, and keeps the Python callback alive for the duration of your session.

Layout

```
from bokeh.layouts import column, row

widgets = row(sel_x, sel_y, sel_r, sel_c)
layout = column(header, widgets, slider, p)
curdoc().add_root(layout)
```

`curdoc()` is the current Bokeh document (the page). `add_root` puts the layout into it.

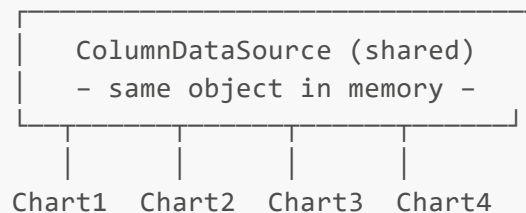
5. Task 3 – Linked Brushing (`p2_brushing.py`)

The problem

A single bubble chart can only show 4 attributes at once. With 28 numeric columns, we're leaving most of the data invisible. Four charts in a 2x2 grid lets us show up to 16 attributes simultaneously.

But when we spot an interesting cluster in Chart 1, how do we know which countries those are in Chart 2? This is the **multiple views problem**, solved by **linked brushing**.

The key insight: one shared source



All four chart renderers point to the **same** `ColumnDataSource` object. Bokeh's selection model lives in `source.selected.indices` (a list of row indices). Changing it triggers a re-render of every renderer that references the source.

Selection appearance

```
cr = p.circle(
    ...
    selection_fill_alpha = 1.0,      # selected: fully opaque
```

```

selection_line_color    = "black",      # selected: black border
selection_line_width    = 1.5,
nonselection_fill_alpha = 0.15,         # others: fade out
nonselection_line_alpha = 0.1,
source = source,           # SAME object in all charts
)

```

When the Box Select tool marks rows as selected in Chart 1, Charts 2-4 see the same `selected.indices` → they fade out non-selected bubbles and highlight selected ones automatically.

Box Select tool

```
p = figure(tools="pan,wheel_zoom,box_select,reset,save")
```

`box_select` lets the user draw a rectangle. All circles whose centers fall inside get added to `selected.indices`.

Per-chart widgets

Each chart has its own four `Select` dropdowns and a scale `Slider`. The callbacks update `cr.glyph.x`, `cr.glyph.y`, etc. directly (property of the glyph renderer), rather than rebuilding `source.data`, because the shared source must stay intact for brushing to work.

gridplot

```

from bokeh.layouts import gridplot

grid = gridplot(
    [[panels[0], panels[1]],
     [panels[2], panels[3]]],
    merge_tools=False,
)

```

`merge_tools=False` keeps each chart's toolbar separate, so the user can use Box Select on one chart without affecting others.

6. Task 4 – Details on Demand (`p2_tooltip.py`)

The information design principle

Shneiderman's mantra:

Overview first → Zoom & filter → Details on demand

The bubble chart gives the overview. Brushing is zoom & filter.

The tooltip provides **details on demand** – extra information that would clutter the chart if always displayed.

Rich tooltip (all 28 attributes)

```
def all_tooltips(numeric_cols):
    tips = [("Country", "@country")]
    for col in numeric_cols:
        tips.append((col, f"@{{{col}}}{0,0.00}"))
    return tips

hover = HoverTool(
    renderers = [cr],
    tooltips = all_tooltips(NUMERIC_COLS),
    point_policy = "follow_mouse", # tooltip follows cursor
)
```

Note the double-brace syntax `@{{{col}}}` in an f-string:

- Outer `{}` = f-string interpolation → inserts column name
- Inner `{{}}` = literal `{}` in the final string → Bokeh format spec

Result: `@{GDP per capita}{0,0.00}` which Bokeh renders as the value from the `GDP per capita` column, formatted with commas and 2 decimals.

Cross-chart hover highlight via CustomJS

Python callbacks are too slow for hover events (every mouse move).

The solution is a `CustomJS` – JavaScript that runs directly in the browser with zero network latency:

```
from bokeh.models import CustomJS

highlight_js = CustomJS(
    args={"source": source, "renderers": circle_renderers},
    code="""
        const idx = cb_data.index.indices; // hovered row indices
        for (let r of renderers) {
            r.data_source.selected.indices = idx.slice();
        }
    """
)
```

`cb_data.index.indices` is the list of data indices under the cursor, provided by Bokeh's hover inspection mechanism.

`.slice()` creates a shallow copy so each chart's selection is independent (avoiding reference aliasing bugs).

Why hover triggers `selection_*` style

When `selected.indices` is set (even to a single-element array), Bokeh applies `selection_fill_alpha = 1.0` and `selection_line_color = "black"` to those circles and `nonselection_fill_alpha = 0.15` to all others.

This creates the "spotlight" effect: the hovered country stands out across all four charts simultaneously.

7. Information Visualization Theory (Exam Preparation)

7.1 Marks and Channels (Bertin, 1967)

Marks are geometric primitives: points, lines, areas.

Channels are visual properties used to encode data:

Channel	Type	Best for
Position (x, y)	Quantitative	Any numeric data
Size / Area	Quantitative	Magnitudes
Color hue	Categorical	Nominal categories
Color saturation/luminance	Quantitative	Ordered data
Shape	Categorical	Small sets of categories
Orientation	Quantitative	Angles, directions

Cleveland & McGill (1984) accuracy ranking (most → least accurate):

1. Position on common scale
2. Position on non-aligned scale
3. Length, direction, angle
4. Area
5. Volume, color saturation
6. Color hue, shape

Implication: encode the most important variable as position (x or y), not color or size.

7.2 The Bubble Chart as a Multi-Channel Design

The bubble chart maps four variables using channels ranked 1, 1, 4, 5 in Cleveland & McGill's hierarchy:

- x → position (rank 1)
- y → position (rank 1)
- area → area (rank 4) – less accurate, but acceptable for population
- color → saturation (rank 5) – least accurate; good only for approximate values

This is why we also provide an exact tooltip – color and size give approximate impressions; the tooltip gives precise values.

7.3 Preattentive Features

Some visual properties are processed **before conscious attention** – they "pop out" instantly. Examples:

- Color (hue)
- Orientation
- Size
- Motion

Our selected bubbles use a black outline (`selection_line_color="black"`) because outline contrast is a preattentive feature – the selected country immediately stands out without the user needing to search.

7.4 Interaction Taxonomy (Shneiderman, 1996)

Type	Description	Our implementation
Select	Mark items of interest	Box Select brushing
Explore	Examine different subsets	Pan / Zoom tools
Reconfigure	Change spatial arrangement	Dropdown to swap axes
Encode	Change visual representation	Dropdown to change color attribute
Abstract/Elaborate	Show overview or detail	Hover tooltip = elaborate
Filter	Show only items meeting criteria	Radius slider
Connect	Reveal relationships between views	Linked brushing

7.5 The Data-Ink Ratio (Tufte)

Maximize the data-ink ratio = data ink / total ink used.

Non-data ink is ink (pixels) that conveys no information: chartjunk, heavy grid lines, redundant labels.

Our design follows this:

- `line_color="white"` for bubble borders (minimal, just separates bubbles)
- No heavy grid (Bokeh default grid is pale)
- Color bar conveys real data (birth rate scale)

7.6 Linked Views – Why They Work

Each chart reveals a **2D projection** of a high-dimensional dataset.

A projection compresses many dimensions into 2, inevitably losing information.

Multiple linked views collectively recover more of the lost structure.

Brushing makes patterns **cross-view coherent**: a cluster that appears in GDP-vs-Life-expectancy space can be immediately located in Inflation-vs-Growth space, revealing whether the economic pattern matches the demographic one.

7.7 Scalability

Concern	Problem	Solution used
Overplotting	Bubbles overlap	<code>fill_alpha=0.7</code>
Clutter from many attributes	Tooltip too long	Show all on hover, not on chart
Slow hover callbacks	Lag on mouse move	<code>CustomJS</code> (browser-side)
Large dataset (many countries)	Visual noise	Box Select to isolate subset

7.8 Color Design

Sequential palette (e.g., `YlOrRd`):

- One hue family, monotonically varying luminance.
- Use for ordered data with a natural zero or minimum (e.g., birth rate).

Diverging palette (e.g., `RdBu`):

- Two hues from a neutral midpoint.
- Use when data has a meaningful center (e.g., GDP growth: negative vs positive).

Categorical palette (e.g., `Category10`):

- 10 perceptually distinct hues.
- Use for nominal/categorical data (e.g., continent).

Perceptual uniformity: In a perceptually uniform palette, equal numeric steps produce equal perceived color differences. `Viridis` and `Plasma` are perceptually uniform; `Jet` (rainbow) is not (it misleads by creating false boundaries).

8. How to Run Everything

Prerequisites

```
pip install bokeh pandas numpy matplotlib
```

File structure

```
homework2/
├── factbook_-_factbook.csv      # raw dataset (upload)
```

```

└─ eda_and_preparation.py      # run first → generates factbook_clean.csv
└─ factbook_clean.csv         # generated by EDA script
└─ p2_bubbles.py              # Task 1 → p2_bubbles.html
└─ p2_widgets.py              # Task 2 → bokeh serve
└─ p2_brushing.py             # Task 3 → bokeh serve
└─ p2_tooltip.py              # Task 4 → bokeh serve
└─ Homework2_Solution.ipynb   # complete notebook

```

Run order

```

# Step 1: clean the data (must be first)
python eda_and_preparation.py

# Step 2: Task 1 (standalone HTML)
python p2_bubbles.py
# → open p2_bubbles.html in your browser

# Step 3-5: Tasks 2-4 (Bokeh server apps)
bokeh serve --show p2_widgets.py
bokeh serve --show p2_brushing.py
bokeh serve --show p2_tooltip.py

```

Tip: Make sure `factbook_clean.csv` is in the **same directory** as the Python scripts, or adjust the `DATA_PATH` variable at the top of each file.

Quick Reference: Bokeh Cheatsheet

```

# Figure
p = figure(width=900, height=600, title="...",
           tools="pan,wheel_zoom,box_select,reset,save")

# Glyphs
p.circle(x, y, radius=r, radius_units="screen",
         fill_color={"field": "col", "transform": mapper},
         fill_alpha=0.7, source=source)

# Color mapper + bar
mapper = LinearColorMapper(palette=YlOrRd9, low=0, high=50)
p.add_layout(ColorBar(color_mapper=mapper, title="label"), "right")

# Hover tooltip
p.add_tools(HoverTool(tooltips=[("Label", "@column{0.00}")]))

# Widgets
sel = Select(title="X axis", value="GDP per capita", options=[...])
sld = Slider(title="Scale", value=1.0, start=0.1, end=5.0, step=0.1)
sel.on_change("value", my_callback)

```

```
# Layout
from bokeh.layouts import column, row, gridplot
layout = column(row(sel, sld), p)
curdoc().add_root(layout)

# Linked brushing - use the SAME source in multiple figures
source = ColumnDataSource(data)
p1.circle(..., source=source)
p2.circle(..., source=source) # selection in p1 reflects in p2

# CustomJS
from bokeh.models import CustomJS
cb = CustomJS(args={"src": source}, code="console.log(src.data);")
```